

Mirna Jeannette Reyes Jiménez

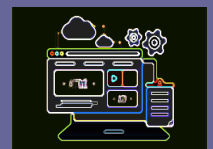
Junio 2025



Práctica 01 - Flynn's taxonomy

Materia: computación paralela

Carrera: Tecnologías de Software



Práctica 01 - Flynn's taxonomy

Suban cuatro códigos de ejemplo en lenguaje C, donde cada uno represente una de las cuatro arquitecturas del **teorema de Flynn**: **SISD**, **MISD**, **SIMD** y **MIMD**.

Pueden elegir el ejemplo que consideren más adecuado, siempre y cuando refleje correctamente los conceptos vistos en clase. Además, deberán elaborar un documento en **formato PDF** que funcione como documentación, donde expliquen cada uno de los cuatro ejemplos implementados, describiendo su funcionamiento y cómo se relaciona con el modelo correspondiente.

Puntos Extras.

Cargar archivos .c a él repositorio en GitHub utilizando tu rama personal.

Entregables:

- **4 archivos en C** (.c), cada uno representando una de las variantes del teorema de Flynn.
- **1 documento en PDF**, donde se explique detalladamente cada ejemplo y su relación con el modelo de Flynn.

Taxonomía de Flynn

La Taxonomía de Flynn, que clasifica las arquitecturas de computadoras según cómo manejan las instrucciones y los datos.

SISD (Single Instruction, Single Data):

Un solo procesador ejecuta una única secuencia de instrucciones en un único flujo de datos. Es la arquitectura secuencial tradicional, la que usas en tu día a día en una computadora sin paralelismo explícito.

Un programa C simple que suma dos números, calcula un factorial, etc. (cualquier cosa secuencial).

Un programa simple que realiza una operación básica.

SIMD (Single Instruction, Multiple Data):

Un solo procesador (o un conjunto de elementos de procesamiento) ejecuta la *misma instrucción* simultáneamente en *múltiples flujos de datos*. Ideal para operaciones vectoriales o en matrices.

Procesadores vectoriales, GPUs (Unidades de Procesamiento Gráfico), instrucciones SIMD en CPUs modernas (SSE, AVX, NEON).

Podemos simular esto usando un bucle que aplique la misma operación a elementos de un arreglo, aunque no será paralelismo *verdadero* a nivel de hardware, representará el concepto de aplicar una instrucción a muchos datos. Para un ejemplo más cercano al paralelismo real, podríamos mencionar el concepto y simularlo.

Opción 1 (Simple): Un bucle que suma un valor a cada elemento de un array.

Opción 2 (Más representativa): Usar OpenMP (si ya lo vieron o pueden investigar un poco) para hacer un bucle for paralelo donde cada hilo aplica la misma instrucción a diferentes partes del array. Para empezar, la Opción 1 es suficiente para ilustrar el concepto.

MISD (Multiple Instruction, Single Data):

Múltiples procesadores ejecutan *diferentes instrucciones* en el *mismo flujo de datos*. Es la menos común en la práctica y, a menudo, difícil de encontrar ejemplos puros. Se asocia a veces con sistemas tolerantes a fallos (donde múltiples procesadores verifican el mismo dato de diferentes maneras) o arquitecturas de flujo de datos (aunque estas son bastante teóricas o de nicho).

Pipeline de procesador, algunos tipos de sistemas de control de vuelo que hacen cálculos redundantes sobre los mismos datos.

Esto es lo más tricky de representar limpiamente. Podríamos simularlo con varios hilos (usando POSIX threads o OpenMP sections) que procesan el *mismo dato* a través de *diferentes funciones/instrucciones*.

Podríamos tener una variable global (el "Single Data") y varios hilos que aplican operaciones distintas (sumar, multiplicar, restar, etc.) sobre esa misma variable, quizá en una secuencia específica.

MIMD (Multiple Instruction, Multiple Data):

Múltiples procesadores ejecutan *diferentes instrucciones* en *diferentes flujos de datos*. Es la arquitectura paralela más flexible y común en la actualidad, abarcando desde computadoras multi-core (con OpenMP o pthreads) hasta clusters distribuidos (con MPI).

CPUs multi-core, clusters de supercomputadoras.

Utilizaremos **OpenMP** para demostrar esto, ya que permite fácilmente la creación de múltiples hilos que pueden realizar tareas independientes en diferentes partes de los datos. Si OpenMP no es una opción, podríamos simularlo con funciones separadas, pero OpenMP es ideal para esto.

Codigo_01.c - SISD (Single Instruction, Single Data)

Este código **SISD** es un ejemplo clásico de computación secuencial. Un solo flujo de instrucciones (main es el único hilo de ejecución) opera sobre un único flujo de datos (las variables numero1, numero2, resultado). El procesador ejecuta la suma paso a paso, una instrucción a la vez, sobre estos datos específicos.

```
Codigo_01.c*  Codigo_04.c  Codigo_03.c  Codigo_02.c
1  #include <stdio.h> // Necesario para funciones de entrada/salida como printf
2
3  int main() {
4      // Declaración e inicialización de variables (Single Data)
5      int numero1 = 10;
6      int numero2 = 5;
7      int resultado;
8
9      // Operación (Single Instruction)
10     resultado = numero1 + numero2;
11
12     // Impresión del resultado
13     printf("SISD (Single Instruction, Single Data) \n");
14     printf("Numero 1: %d\n", numero1);
15     printf("Numero 2: %d\n", numero2);
16     printf("Resultado de la suma: %d\n", resultado);
17     printf("\n");
18
19     return 0; // Indica que el programa terminó correctamente
20 }
```

```
MINGW64:/c:/Users/mirna/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025
mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJean
netteReyesJimenez)
$ ls
Codigo_01.c  Codigo_02.c  Codigo_03.c  Codigo_04.c  README.md

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJean
netteReyesJimenez)
$ gcc Codigo_01.c -o Codigo_01

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJean
netteReyesJimenez)
$ ./Codigo_01
SISD (Single Instruction, Single Data)
Numero 1: 10
Numero 2: 5
Resultado de la suma: 15

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJean
netteReyesJimenez)
$ |
```

Codigo_02.c - SIMD (Single Instruction, Multiple Data)

Aquí simularemos la aplicación de la misma operación a múltiples elementos de un arreglo. Usaremos OpenMP para lograr paralelismo real.

Este programa SIMD utiliza OpenMP para sumar elementos de dos grandes arreglos. La directiva `#pragma omp parallel for` indica que el bucle `for` debe ejecutarse en paralelo, con diferentes hilos de ejecución trabajando en diferentes partes del arreglo. La clave es que todos los hilos ejecutan la misma instrucción de suma (`result_array[i] = array_a[i] + array_b[i];`) pero lo hacen sobre diferentes elementos del arreglo (múltiples datos) simultáneamente.

```
Codigo_01.c  X Codigo_04.c  Codigo_03.c  Codigo_02.c  X
1  #include <stdio.h>
2  #include <stdlib.h> // Para malloc y free
3  #include <omp.h>    // Necesario para OpenMP
4
5  #define SIZE 1000 // Tamaño del arreglo
6
7  int main() { // <--- ASEGURATE QUE LA FUNCION MAIN ABRA AQUI
8      // Declaración y asignación de memoria para los arreglos (Multiple Data)
9      int *array_a = (int *)malloc(sizeof(int) * SIZE);
10     int *array_b = (int *)malloc(sizeof(int) * SIZE);
11     int *result_array = (int *)malloc(sizeof(int) * SIZE);
12
13     // Inicialización de los arreglos
14     for (int i = 0; i < SIZE; i++) {
15         array_a[i] = i;
16         array_b[i] = i * 2;
17     }
18
19     printf("--- SIMD (Single Instruction, Multiple Data) ---\n");
20     printf("Realizando la suma de elementos de dos arreglos de tamaño %d...\n", SIZE);
21
22     // Bloque paralelo de OpenMP
23     // La misma instrucción (suma) se aplica a multiples datos (elementos del arreglo)
24     #pragma omp parallel for // Directiva OpenMP: divide el bucle entre los hilos
25     for (int i = 0; i < SIZE; i++) {
26         result_array[i] = array_a[i] + array_b[i]; // Misma instrucción: suma
27     }
28
29     // Imprimir algunos resultados para verificar (opcional, solo para depuración)
30     printf("Primeros 5 resultados:\n");
31     for (int i = 0; i < 5; i++) {
32         printf("result_array[%d] = %d\n", i, result_array[i]);
33     }
```

```

34     printf("...\n");
35     printf("Ultimos 5 resultados:\n");
36     for (int i = SIZE - 5; i < SIZE; i++) {
37         printf("result_array[%d] = %d\n", i, result_array[i]);
38     }
39
40     printf("-----\n");
41
42     // Liberar memoria asignada
43     free(array_a);
44     free(array_b);
45     free(result_array);
46
47     return 0;
48 }

```

```

MINGW64:/c:/Users/mirna/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025
etteReyesJimenez)
$ gcc -fopenmp Codigo_02.c -o codigo_02

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeanr
etteReyesJimenez)
$ ./Codigo_02
--- SIMD (Single Instruction, Multiple Data) ---
Realizando la suma de elementos de dos arreglos de tamaño 1000...
Primeros 5 resultados:
result_array[0] = 0
result_array[1] = 3
result_array[2] = 6
result_array[3] = 9
result_array[4] = 12
...
Últimos 5 resultados:
result_array[995] = 2985
result_array[996] = 2988
result_array[997] = 2991
result_array[998] = 2994
result_array[999] = 2997
-----

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeanr
etteReyesJimenez)
$

```

Codigo_03.c - MISD (Multiple Instruction, Single Data)

simularemos tener diferentes "procesadores" (hilos/funciones) aplicando distintas instrucciones a un único dato. Usaremos OpenMP sections para que diferentes hilos ejecuten bloques de código distintos sobre el mismo dato.

Este ejemplo **MISD** utiliza omp parallel sections para que múltiples hilos operen sobre una única variable global, shared_data. Cada #pragma omp section define un bloque de código que puede ser ejecutado por un hilo diferente. Así, múltiples hilos ejecutan **diferentes instrucciones** (resta, multiplicación, división) sobre el **mismo dato** (shared_data). Es una simulación conceptual, y en un escenario real MISD a menudo implica redundancia para verificación o procesamiento en un pipeline. La salida del valor final de shared_data puede variar en cada ejecución debido a la **condición de carrera**, ya que los hilos modifican el mismo dato sin una sincronización explícita de acceso.

```
Codigo_01.c  Codigo_04.c  Codigo_03.c  Codigo_02.c
1  #include <stdio.h>
2  #include <omp.h> // Necesario para OpenMP
3
4  // Definimos una variable global que sera nuestro "Single Data"
5  int shared_data = 100;
6
7  int main() {
8      printf("--- MISD (Multiple Instruction, Single Data) ---\n");
9      printf("Dato inicial: %d\n", shared_data);
10
11     // Bloque paralelo de OpenMP con secciones
12     // Multiples hilos ejecutaran diferentes bloques de codigo (Multiples Instrucciones)
13     // sobre el mismo dato (shared_data).
14     #pragma omp parallel sections
15     {
16         #pragma omp section // Primera seccion: Operacion de resta
17         {
18             // Declara tid dentro de esta seccion, sera privado para el hilo que la ejecute
19             int tid = omp_get_thread_num();
20             // Nota: Aquí los hilos compiten por shared_data, el orden no es garantizado.
21             shared_data = shared_data - 10; // Instruccion 1: resta
22             printf("Hilo %d: Despues de restar 10, shared_data = %d\n", tid, shared_data);
23         }
24
25         #pragma omp section // Segunda seccion: Operacion de multiplicacion
26         {
27             // Declara tid dentro de esta seccion
28             int tid = omp_get_thread_num();
29             shared_data = shared_data * 2; // Instruccion 2: multiplicacion
30             printf("Hilo %d: Despues de multiplicar por 2, shared_data = %d\n", tid, shared_data);
31         }
32     }
```



```

32
33     #pragma omp section // Tercera seccion: Operacion de division
34     {
35         // Declara tid dentro de esta seccion
36         int tid = omp_get_thread_num();
37         shared_data = shared_data / 5; // Instruccion 3: division
38         printf("Hilo %d: Despues de dividir por 5, shared_data = %d\n", tid, shared_data);
39     }
40 } // Fin del bloque parallel sections
41
42 printf("Dato final despues de multiples instrucciones: %d\n", shared_data);
43 printf("-----\n");
44
45 return 0;
46 }

```

MINGW64:/c/Users/mirna/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025

```

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeannetteReyesJimenez)
$ gcc -fopenmp Codigo_03.c -o Codigo_03

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeannetteReyesJimenez)
$ ./Codigo_03
--- MISD (Multiple Instruction, Single Data) ---
Dato inicial: 100
Hilo 13: Despues de restar 10, shared_data = 90
Hilo 15: Despues de multiplicar por 2, shared_data = 180
Hilo 14: Despues de dividir por 5, shared_data = 36
Dato final despues de multiples instrucciones: 36
-----

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeannetteReyesJimenez)
$

```

Codigo_04.c - MIMD (Multiple Instruction, Multiple Data)

Este es el modelo más flexible y común para el paralelismo, donde diferentes hilos o procesos pueden hacer cosas completamente distintas en diferentes conjuntos de datos.

Este programa **MIMD** demuestra el paralelismo más flexible. Utilizamos OpenMP para crear dos hilos de ejecución. Cada hilo (tid == 0 y tid == 1) ejecuta una **instrucción diferente** (una función distinta: tarea_hilo_0 y tarea_hilo_1) y opera sobre un **conjunto de datos diferente** (data_set_1 y data_set_2). Esto refleja la capacidad de los sistemas MIMD de realizar múltiples tareas diversas simultáneamente, cada una con su propio flujo de instrucciones y datos.

```
Codigo_01.c  Codigo_04.c  Codigo_03.c  Codigo_02.c
1  #include <stdio.h>
2  #include <omp.h> // Necesario para OpenMP
3  #include <unistd.h> // Para sleep (para simular trabajo)
4
5  // Función para el Hilo 0: procesa el primer conjunto de datos
6  void tarea_hilo_0(int* data, int size) {
7      printf("Hilo %d: Procesando Datos 1 (sumando 10 a cada elemento)...\n", omp_get_thread_num());
8      for (int i = 0; i < size; i++) {
9          data[i] += 10; // Diferente instruccion 1
10         usleep(100); // Pequeña pausa para simular trabajo
11     }
12     printf("Hilo %d: Datos 1 procesados.\n", omp_get_thread_num());
13 }
14
15 // Función para el Hilo 1: procesa el segundo conjunto de datos
16 void tarea_hilo_1(int* data, int size) {
17     printf("Hilo %d: Procesando Datos 2 (multiplicando por 2 cada elemento)...\n", omp_get_thread_num());
18     for (int i = 0; i < size; i++) {
19         data[i] *= 2; // Diferente instruccion 2
20         usleep(100); // Pequeña pausa para simular trabajo
21     }
22     printf("Hilo %d: Datos 2 procesados.\n", omp_get_thread_num());
23 }
24
25 int main() {
26     // Definimos diferentes conjuntos de datos (Multiple Data)
27     int data_set_1[5] = {1, 2, 3, 4, 5};
28     int data_set_2[5] = {10, 20, 30, 40, 50};
29
30     printf("--- MIMD (Multiple Instruction, Multiple Data) ---\n");
31 }
```

```

33  #pragma omp parallel num_threads(2) // Creamos 2 hilos
34  {
35      int tid = omp_get_thread_num(); // Obtener el ID del hilo
36
37      if (tid == 0) {
38          // Hilo 0 ejecuta una instruccion diferente en data_set_1
39          tarea_hilo_0(data_set_1, 5);
40      } else if (tid == 1) {
41          // Hilo 1 ejecuta otra instruccion diferente en data_set_2
42          tarea_hilo_1(data_set_2, 5);
43      }
44  } // Fin del bloque parallel
45
46  printf("\nResultados finales:\n");
47  printf("Data Set 1: ");
48  for (int i = 0; i < 5; i++) {
49      printf("%d ", data_set_1[i]);
50  }
51  printf("\n");
52
53  printf("Data Set 2: ");
54  for (int i = 0; i < 5; i++) {
55      printf("%d ", data_set_2[i]);
56  }
57  printf("\n");
58  printf("-----\n");
59
60  return 0;
61  }

```

```

MINGW64:/c:/Users/mirna/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025
mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeannetteReyesJimenez)
$ gcc -fopenmp Codigo_04.c -o Codigo_04

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeannetteReyesJimenez)
$ ./Codigo_04
--- MIMD (Multiple Instruction, Multiple Data) ---
Hilo 0: Procesando Datos 1 (sumando 10 a cada elemento)...
Hilo 1: Procesando Datos 2 (multiplicando por 2 cada elemento)...
Hilo 0: Datos 1 procesados.
Hilo 1: Datos 2 procesados.

Resultados finales:
Data Set 1: 11 12 13 14 15
Data Set 2: 20 40 60 80 100
-----

mirna@Mirna MINGW64 ~/OneDrive/Desktop/repositorio Mirna/Ceti-PP-2025 (MirnaJeannetteReyesJimenez)
$

```



Conclusión

La realización de esta práctica ha permitido una comprensión profunda y aplicada de la Taxonomía de Flynn, un marco fundamental para clasificar las arquitecturas de computadoras en función de cómo gestionan los flujos de instrucciones y datos. A través de la implementación de ejemplos en lenguaje C para los modelos SISD, SIMD, MISD y MIMD, se han podido visualizar los principios operativos de cada configuración.

El ejercicio demostró que, mientras SISD representa la computación secuencial tradicional, los modelos SIMD y MIMD son esenciales para el paralelismo moderno. SIMD, ejemplificado por la operación vectorial sobre múltiples datos, subraya la eficiencia en tareas repetitivas a gran escala. Por otro lado, MIMD, la arquitectura más flexible y ubicua hoy en día, ilustra la capacidad de múltiples unidades de procesamiento para ejecutar tareas diversas y complejas de forma concurrente, cada una con su propio flujo de instrucciones y datos. Aunque MISD presenta desafíos conceptuales en su implementación práctica, el ejemplo proporcionado ayudó a ilustrar la ejecución de distintas instrucciones sobre un único flujo de datos, destacando las consideraciones sobre la coherencia y sincronización.

En definitiva, entender la Taxonomía de Flynn no solo es crucial para analizar el rendimiento y la escalabilidad de los sistemas informáticos, sino que también es un paso indispensable para el diseño y desarrollo de aplicaciones que aprovechen eficazmente la computación paralela, un pilar fundamental en la era actual de la información."

